CW VENTURES

Data Room Platform

Productization Specification

Prepared for
Cameron Wiese
Date
February 2026
Status
Draft — For Review
Scope
3-phase productization of investor data room

This document provides complete technical specifications for transforming the CW Ventures investor data room into a repeatable, multi-client deployment platform. Each phase includes file-by-file implementation details, time estimates, and automation recommendations.

Table of Contents

Phase 1: Extract Configuration from Code 3

Phase 2: Deployment Pipeline & GitHub Workflow 10

Phase 3: Hardening for External Use 16

Quickstart: Fastest Path to Test Deployment 21

Appendix A: Hardcoded Values Inventory 25

Appendix B: Client Onboarding Checklist 27

Phase 1: Extract Configuration from Code

Goal: Separate all company-specific content, branding, and contact information from reusable platform code so that new deployments require only a configuration change, not code edits.

Estimated effort: 8–12 hours

Prerequisite: None — this is the foundation for everything else.

1.1 Create the Site Configuration File

Create a new file at the project root that holds every company-specific value. Every component that currently hardcodes company info will import from this file instead.

New file: /config/site.js

```
// /config/site.js
// All company-specific values for this deployment

export const siteConfig = {
  // ── Company ──
  companyName: "CW Ventures",
  dealName: "Puget Sound Therapeutics",
  dealSlug: "pst",
  roundType: "Seed Round · SPV Data Room",
  copyrightYear: "2026",

  // ── Founder ──
  founderName: "Cameron Wiese",
  founderEmail: "camwiese@gmail.com",
  founderPhone: "360-318-4480",
  calendarUrl: "https://calendar.app.google/asdgEY3xfoLCNyWJ9",
  signatureImage: "/images/signature.png",

  // ── Branding ──
  theme: {
    primaryDark: "#1a3a2a",
    primary: "#2d6144",
    primaryLight: "#e6f5ed",
    accent: "#3a7d59",
    background: "#fdfbf7",
  },

  // ── Tabs ──
  tabs: [
    { id: "overview", label: "Executive Summary",
      short: "Summary", weight: 1 },
    { id: "memo", label: "Deal Memo",
      short: "Memo", weight: 2 },
    { id: "terms", label: "Deal Terms",
      short: "Terms", weight: 3 },
    // ... etc
  ],

  // ── Financial Model ──
  model: {
```

```
    safeCap: 25000000,
    roundSize: 5000000,
    seriesARaise: 35000000,
    seriesAPreMoney: 100000000,
    optionPool: 15,
    carry: 0.20,
    exitYear: 2029,
    defaultInvestAmount: 50000,
  },
};
```

Files that must import from site.js

The following table shows every file with hardcoded values and the specific values that need to be replaced with config references.

File	Hardcoded Values	Config Key
app/pst/page.js	"CW Ventures" (sidebar title)	siteConfig.companyName
	"Puget Sound Therapeutics" (subtitle)	siteConfig.dealName
	"Seed Round · SPV Data Room"	siteConfig.roundType
	Calendar URL (sidebar CTA)	siteConfig.calendarUrl
	Phone: 360-318-4480	siteConfig.founderPhone
	© 2026 CW Ventures	siteConfig.copyrightYear + companyName
	useTracker("pst", ...)	siteConfig.dealSlug
app/layout.js	"CW Ventures" (metadata title)	siteConfig.companyName
	"Private investor data room"	siteConfig.companyName
	cw-ventures.vercel.app fallback URL	NEXT_PUBLIC_APP_URL env var
components/Shared.js	"Puget Sound Therapeutics" in SectionHeader	siteConfig.dealName
	sms:3603184480 in MobileContactBar	siteConfig.founderPhone
components/CallSection.js	Calendar URL, phone, SMS link	siteConfig.calendarUrl, founderPhone
	"Chat with Cameron"	siteConfig.founderName
components/ OverviewSection.js	Entire letter content	Move to /content/overview.md
	Signature image path	siteConfig.signatureImage
	camwiese@gmail.com, phone	siteConfig.founderEmail, founderPhone

components/ ModelSection.js	safeCap: 25000000	siteConfig.model.safeCap
	seriesARaise: 35000000	siteConfig.model.seriesARaise
	seriesAPreMoney: 100000000	siteConfig.model.seriesAPreMoney
	carry: 0.20, optionPool: 15	siteConfig.model.carry, optionPool
	exitYear: 2029	siteConfig.model.exitYear
components/ FAQSection.js	~50 Q&A items (all PST-specific)	Move to /content/faq.json
components/ TermsSection.js	Post-Money SAFE, $25M cap, $5M round	siteConfig.model.*
components/ InterviewSection.js	"Cameron and Daniel" description	Move to /content/interview.md
lib/notifications.js	"CW Ventures <hello@ventures...>"	siteConfig.companyName + env var
	Pacific timezone	siteConfig.timezone (optional)
constants/tabs.js	TAB_ORDER, TAB_WEIGHTS, NAV_ITEMS	Derive from siteConfig.tabs
constants/theme.js	COLORS object (green/cream palette)	Merge with siteConfig.theme
app/api/track/route.js	ALLOWED_DEAL_SLUGS = ["pst"]	siteConfig.dealSlug

1.2 Create the Content Directory

Move all deal-specific prose out of React components and into a /content directory. Components become generic renderers that consume markdown or JSON.

Directory structure

```
/content/
  overview.md          # Founder letter (currently in OverviewSection.js)
  memo.md              # Already exists as markdown
  terms.json           # Round details array for TermsSection
  faq.json             # Q&A array with categories
  science.md           # Already exists as markdown
  interview.md         # Video embed URL + description
  model-scenarios.json # Exit scenarios for ModelSection
```

FAQ content format (faq.json)

```
[
  { "category": "Investment at a Glance",
    "items": [
      { "q": "Why is the risk mispriced?",
        "a": "Markdown-formatted answer..." },
    ]
  }
]
```

Key decision: Markdown is the simplest format for a non-technical founder to edit. You could use a headless CMS later (Sanity, Contentful), but for 5–10 clients, flat files in the repo are perfectly fine and avoid another moving part.

1.3 Refactor Components

Each content component needs to become a generic renderer. Here’s the pattern for each:

OverviewSection.js

Currently 26 lines of hardcoded JSX. Replace with a markdown renderer that reads from /content/overview.md and appends the founder signature block from config.

FAQSection.js

Currently ~150 lines of inline Q&A. Replace the hardcoded cats array with a JSON import from /content/faq.json. The Accordion component is already generic — it just needs data piped in.

ModelSection.js

Currently has 5 hardcoded financial constants on lines 9–14. Replace with imports from siteConfig.model. The scenario arrays on line 40–41 should also come from config or /content/model-scenarios.json.

TermsSection.js

The roundDetails array on lines 5–11 is hardcoded. Move to /content/terms.json or derive from siteConfig.model.

CallSection.js

Hardcoded calendar URL, phone number, and SMS link. Replace all three with siteConfig references. Change the title from “Chat with Cameron” to “Chat with {siteConfig.founderName}”.

Shared.js — SectionHeader

Line that says “Puget Sound Therapeutics” needs to read from siteConfig.dealName.

1.4 Refactor the Tracking API

The /api/track/route.js file has a hardcoded allowlist:

```
const ALLOWED_DEAL_SLUGS = ["pst"];
```

Replace with:

```
import { siteConfig } from "../../../config/site";
const ALLOWED_DEAL_SLUGS = [siteConfig.dealSlug];
```

1.5 Refactor Notifications

lib/notifications.js hardcodes the sender name and email:

```
from: "CW Ventures <hello@ventures.cameronwiese.com>"
```

Replace with:

```
const senderName = siteConfig.companyName;
const senderEmail = process.env.AUTH_SENDER_EMAIL;
from: `${senderName} <${senderEmail}>`
```

1.6 Refactor Layout Metadata

app/layout.js has hardcoded OpenGraph and Twitter metadata. Replace all instances of “CW Ventures” with siteConfig.companyName. Note: since layout.js is a Server Component, you’ll need to ensure siteConfig is importable server-side (it will be, since it’s a plain JS module).

Validation checkpoint

After completing Phase 1, run: npm run build && npm run dev

Verify: all pages render correctly, analytics track, notifications fire.

Then change 3 values in site.js (companyName, dealSlug, founderPhone) and verify they propagate everywhere.

If any hardcoded value survives, grep the codebase: grep -r "CW Ventures" --include="*.js"

Phase 2: Deployment Pipeline & GitHub Workflow

Goal: Create a repeatable, partially automated process for spinning up a new client's data room in under 2 hours.

Estimated effort: 6–8 hours (includes writing the automation script)

Prerequisite: Phase 1 complete.

2.1 Repository Strategy

You have two viable approaches. Here's the decision:

Approach	How It Works	Pros	Cons
Template repo + clone	Keep one GitHub template repo. For each client, create a private repo from the template, customize site.js and /content, deploy.	Simple. Each client is fully independent. Easy to understand.	Updating core platform means cherry-picking into each repo.
Monorepo with config branches	Single repo with a main branch (platform code) and per-client branches that override config/content only.	One place to update core code. Merge main into client branches to propagate updates.	Branch management gets messy after 5+ clients. Merge conflicts on config files.

Recommendation: Template repo + clone

For your scale (5–10 clients), template repo is the right call.
It's simpler to reason about, clients are fully isolated, and the "update propagation" problem is manageable with a simple script that diffs template changes and applies them.
You can always move to a monorepo later if you hit 20+ clients.

2.2 GitHub Setup

Here's the exact sequence:

1. **Convert your current repo to a template.** Go to github.com/camwiese/cw-ventures → Settings → check "Template repository." This lets you create new repos from it with one click.
2. **Rename the template repo.** Rename from cw-ventures to something generic: dataroom-platform or investor-dataroom. This is your source of truth for platform code.
3. **For each new client,** click "Use this template" on GitHub to create a new private repo. Name it by client: dataroom-clientname.
4. **Clone the new repo locally,** customize config/site.js and /content/, commit, push. Vercel auto-deploys.

Alternative: CLI-based cloning

If you prefer automation over the GitHub UI:

```
# Create new client repo from template
gh repo create camwiese/dataroom-acme \
  --template camwiese/dataroom-platform \
  --private

git clone https://github.com/camwiese/dataroom-acme.git
cd dataroom-acme

# Customize config (edit site.js, replace /content/)
# Then push
git add -A && git commit -m 'Configure for Acme Corp'
git push origin main
```

2.3 Per-Client Infrastructure Setup

Each client gets their own isolated stack. Here's every step:

Step 1: Supabase Project

1. Go to supabase.com → New Project. Name it dataroom-clientname.
2. Copy the project URL, anon key, and service role key.
3. Go to SQL Editor, paste the contents of supabase-setup.sql, and run it. This creates all 3 tables (page_views, otp_attempts, high_intent_notifications) with RLS policies.
4. Go to Authentication → Providers → Enable Email (magic link only, disable password).
5. Go to Authentication → URL Configuration: set Site URL to the client's domain (e.g., https://dataroom.acmecorp.com). Add localhost:3000 as an additional redirect URL for testing.
6. Go to Authentication → Email Templates: paste the magic link template from DEPLOY.md, replacing CW Ventures branding with the client's name.

Time: ~20 minutes per client.

Step 2: Resend Email Setup

1. Go to resend.com → Domains → Add the client's sending domain (e.g., mail.acmecorp.com).
2. Add the 3 DNS records Resend provides (SPF, DKIM, DMARC) to the client's DNS.
3. Wait for verification (usually 5–30 minutes).
4. Create an API key scoped to this domain.

Time: ~15 minutes plus DNS propagation wait.

> **Alternative: Use your own sending domain**
> If the client doesn't want to configure DNS, you can send from your own domain (e.g., hello@dataroom.cameronwiese.com) with their company name in the From field. This is faster but slightly less professional. Good enough for early clients.

Step 3: Vercel Project

1. Go to vercel.com → New Project → Import from the client's GitHub repo.
2. Set the Framework Preset to Next.js.
3. Add environment variables (see table below).
4. Deploy. Vercel builds and deploys automatically.
5. Go to Settings → Domains → Add the client's custom domain.

6. Add Vercel's CNAME record to the client's DNS.

Environment Variable	Value	Notes
NEXT_PUBLIC_SUPABASE_URL	https://xxx.supabase.co	From Step 1
NEXT_PUBLIC_SUPABASE_ANON_KEY	eyJ...	From Step 1
SUPABASE_SERVICE_ROLE_KEY	eyJ...	From Step 1 (server-only)
GP_EMAIL	founder@acmecorp.com	Client's email for admin access
RESEND_API_KEY	re_xxx	From Step 2
AUTH_SENDER_EMAIL	hello@mail.acmecorp.com	Verified in Resend
NEXT_PUBLIC_APP_URL	https://dataroom.acmecorp.com	Client's domain
NOTIFICATION_EXCLUDE_EMAILS	founder@acmecorp.com	Prevent self-notifications
ANALYTICS_EXCLUDE_EMAILS	founder@acmecorp.com	Exclude from analytics

Time: ~20 minutes per client.

Step 4: Domain Configuration

The client needs to add DNS records for two things:

- **Vercel hosting:** CNAME record pointing their subdomain (e.g., dataroom.acmecorp.com) to cname.vercel-dns.com. Vercel handles SSL automatically.
- **Email sending:** SPF, DKIM, and DMARC records for their sending domain (from Resend). Only needed if sending from their domain.

Time: ~10 minutes of your time, plus up to 48 hours for DNS propagation (usually under 1 hour).

2.4 The Setup Automation Script

To reduce the manual steps above, create a setup script that automates as much as possible. The script won't fully eliminate manual work (Supabase and Vercel don't have great CLI support for project creation), but it can handle the repetitive parts.

New file: /scripts/setup-client.sh

```
#!/bin/bash
# Usage: ./scripts/setup-client.sh <client-slug>

CLIENT_SLUG="$1"
TEMPLATE_REPO="camwiese/dataroom-platform"

echo "=== Creating repo for $CLIENT_SLUG ==="
gh repo create camwiese/dataroom-$CLIENT_SLUG \
  --template $TEMPLATE_REPO --private --clone

cd dataroom-$CLIENT_SLUG

echo "=== Customize config/site.js now ==="
echo "Then run: git add -A && git commit && git push"
echo ""
echo "=== Manual steps remaining ==="
echo "1. Create Supabase project, run schema"
echo "2. Create Vercel project from this repo"
echo "3. Set env vars in Vercel dashboard"
echo "4. Configure client DNS records"
echo "5. Test auth flow end-to-end"
```

This is intentionally simple. You could automate Supabase project creation with their Management API and Vercel with their CLI, but for 5–10 clients, the manual steps take 20 minutes and the automation would take hours to build and maintain.

2.5 Propagating Platform Updates

When you fix a bug or add a feature to the platform, you need to get it into each client's repo. Here's the workflow:

1. **Make the change in your template repo** (dataroom-platform). Test it there.
2. **For each client repo,** add the template as a remote and cherry-pick or merge:

```
cd dataroom-clientname
git remote add platform https://github.com/camwiese/dataroom-platform.git
git fetch platform
git cherry-pick <commit-hash>
git push origin main  # Vercel auto-deploys
```

3. **If the change touches config/site.js or /content/,** you'll get a merge conflict. Resolve it by keeping the client's config and taking the platform's structural changes.

Time per client: 5–10 minutes for a clean cherry-pick, 15–20 minutes if there's a merge conflict.

Automation opportunity

Once you have 5+ clients, write a script that loops through all client repos, fetches from platform, and attempts a cherry-pick. It can flag conflicts for manual resolution. This turns a 30-minute task into a 5-minute one.

2.6 Complete Deployment Checklist

Print this for each new client:

#	Step	Owner	Time	Done?
1	Receive client content package (see Appendix B)	Client	1–3 days	☐
2	Create GitHub repo from template	You	2 min	☐
3	Customize config/site.js	You	15 min	☐
4	Replace /content/ files with client content	You	30 min	☐
5	Replace /public/images/ (logo, favicon, signature)	You	10 min	☐
6	Create Supabase project + run schema	You	15 min	☐
7	Configure Supabase auth (magic link, URLs, email template)	You	10 min	☐
8	Set up Resend domain (or use yours)	You	15 min	☐
9	Create Vercel project + set env vars	You	15 min	☐
10	Add client's custom domain to Vercel	You	5 min	☐
11	Client adds DNS records (Vercel CNAME + Resend SPF/DKIM)	Client	10 min	☐
12	Test: magic link login flow	You	5 min	☐
13	Test: all tabs render with client content	You	10 min	☐
14	Test: admin page loads for GP_EMAIL	You	5 min	☐
15	Test: notification emails fire on first visit	You	5 min	☐
16	Test: intent score calculates correctly	You	5 min	☐
17	Client walkthrough call	Both	30 min	☐

Total hands-on time: ~3–4 hours (excluding wait time for DNS and client content).

Phase 3: Hardening for External Use

Goal: Make the platform reliable and polished enough that a non-technical founder can use it confidently during their fundraise without needing your intervention for common issues.

Estimated effort: 8–14 hours

Prerequisite: Phase 1 complete. Phase 2 recommended but not required.

3.1 Authentication Hardening

Magic links are great for investor UX, but they have known edge cases that you currently don't handle gracefully.

Problem: Expired magic links

Supabase magic links expire after 1 hour by default. If an investor opens the email hours later, they see a raw error. Currently your /auth/verify page catches this, but the error message is generic.

Fix: Add a friendly expired-link page that says "This link has expired. Enter your email below to get a new one." with an inline email input that calls /api/auth/request-otp. This keeps the investor in the flow instead of abandoning.

Problem: Corporate email link scanning

Microsoft Outlook Safe Links, Proofpoint, Mimecast, and similar enterprise email security tools pre-fetch links in emails. This "consumes" the magic link before the investor clicks it. You already have a 6-digit code fallback, which is good.

Fix: Make the 6-digit code more prominent in the email template. Currently it's secondary text. Move it to equal prominence: "Click the button below, or enter this code: 123456." Also add a note: "If the button doesn't work, use the code instead."

Problem: Rate limiting visibility

Supabase rate-limits OTP requests (default: 1 per 60 seconds per email). If an investor clicks "Resend" too quickly, the request silently fails.

Fix: Add a cooldown timer on the login page. After sending an OTP, disable the send button for 60 seconds with a countdown: "Resend in 45s." This prevents the investor from thinking the system is broken.

3.2 Admin Dashboard Improvements

Your admin page is functional but built for someone who understands the data. A less technical founder needs more context.

Add a summary header

At the top of the admin page, add cards showing:

- Total unique investors who have accessed the data room
- New investors this week
- Highest intent investor (name + score)
- Average time spent per investor

These already exist in your code (the summary object in admin/page.js computes them), but they're displayed in a table row. Move them to prominent cards at the top.

Add intent score explanation

Below the investor table, add a collapsible "How intent scoring works" section that explains: what the score means, what tab weights are, what "heating up" means, and what the suggested next steps mean. This prevents the founder from asking you "what does 47 mean?" every week.

Add email-based admin login

Currently, admin access is determined by matching the logged-in user's email against GP_EMAIL. This works, but means the founder has to go through the magic link flow just like an investor. Consider adding a separate /admin/login route, or at minimum make the regular login redirect to /admin if the email matches GP_EMAIL.

3.3 Error Handling & Resilience

Tracking API resilience

The /api/track endpoint does fire-and-forget analytics via sendBeacon. If Supabase is temporarily down, those analytics are lost. For your own use this is acceptable; for a paying client, losing investor engagement data is a problem.

Fix (lightweight): Add a simple retry. If the Supabase insert fails, retry once after 2 seconds. If that fails, log the error to a Supabase “failed_events” table (which itself might fail, but at least you’ve tried). This catches transient network issues without building a full queue system.

Build-time content validation

If a client’s faq.json is malformed or a required config field is missing, the app shouldn’t crash at runtime. Add build-time validation in next.config.mjs:

```
// Validate site config at build time
const { siteConfig } = require("./config/site");
const required = ["companyName", "dealSlug", "founderEmail"];
for (const key of required) {
  if (!siteConfig[key]) {
    throw new Error(`Missing siteConfig.${key}`);
  }
}
```

Also validate that /content/faq.json parses correctly and has the expected structure. Fail fast at build time, not at runtime when an investor is trying to view the FAQ.

Notification failure alerting

Currently, notification failures are silently caught. If Resend is down or the API key is wrong, you won’t know until the founder complains that they’re not getting alerts.

Fix: Add a simple health check: on the admin page, show the last successful notification timestamp. If it’s been more than 7 days with active investors and no notifications, show a warning banner.

3.4 Security Review

Your current security posture is good for a single-user deployment. For external clients, tighten a few things:

Area	Current State	Recommended Change
RLS policies	All 3 tables deny anon/authenticated access	Good as-is. Service role only.
Service role key	Used in server-side API routes only	Verify it never appears in client bundle. Add a build check.

CORS	Not explicitly configured (Next.js defaults)	Add explicit CORS headers to /api/track to prevent cross-origin abuse.
Rate limiting	Supabase handles OTP rate limiting	Add rate limiting to /api/track (e.g., max 60 requests/minute per user).
Admin access	Email match against GP_EMAIL env var	Good. Consider adding a secondary admin email for backup access.
Session management	Supabase handles session cookies	Add session timeout (e.g., 30 days). Currently sessions persist indefinitely.
CSP headers	Not configured	Add Content-Security-Policy to next.config.mjs to prevent XSS.

3.5 Testing Checklist

Before deploying any client's data room, run through this test matrix:

Test	Steps	Expected Result
Magic link (fresh)	Enter email → receive email → click link	Redirected to /pst (or deal slug), session created
Magic link (expired)	Wait >1hr, click old link	Friendly error with re-send option
6-digit code	Enter email → enter code from email	Logged in successfully
Rate limit	Send 3 OTPs in 30 seconds	Clear rate limit message, no silent failure
Admin access (GP)	Log in with GP_EMAIL	Redirected to /admin, see investor data
Admin access (non-GP)	Log in with different email, navigate to /admin	Redirected to /pst
Tab tracking	View 3 tabs for 10+ seconds each	page_views table shows 3 records
First-visit notification	Log in as new investor	GP receives "New investor" email
Return-visit notification	Wait 30+ min, log in again	GP receives "is back" email
High-intent notification	Generate 50+ intent score	GP receives "High intent" email
Revoke access	Admin clicks Revoke on an investor	Investor's data deleted, auth removed
Mobile responsive	View on phone-width browser	Bottom nav appears, no horizontal scroll
Content rendering	Check every tab	All markdown/JSON content renders correctly

Quickstart: Fastest Path to Test Deployment

Goal: Go from where you are today to a working test deployment for a fictional client in the shortest possible time, validating the entire pipeline before taking on a real client.

Estimated time: 4–6 hours total, split across 2 sessions.

Session 1: Config Extraction (2–3 hours)

Do the minimum viable version of Phase 1. You don't need to refactor every component perfectly — you need to prove the concept works.

1. **Create config/site.js** with all the values listed in section 1.1. Copy the exact structure.
2. **Update these 5 files only** (the highest-impact changes):
   - app/pst/page.js — Replace sidebar text, calendar URL, phone number, deal slug
   - components/Shared.js — Replace "Puget Sound Therapeutics" in SectionHeader
   - components/CallSection.js — Replace calendar URL, phone, name
   - lib/notifications.js — Replace sender name and email
   - app/api/track/route.js — Replace ALLOWED_DEAL_SLUGS
3. **Move FAQ content to /content/faq.json.** This is the biggest content block. Once it's externalized, you've proven the pattern.
4. **Test locally:** npm run dev. Navigate every tab. Confirm nothing says "CW Ventures" or "PST" (unless that's your test config).
5. **Commit to a new branch:** git checkout -b refactor/config-extraction

Session 2: Test Deployment (2–3 hours)

Spin up a complete fake client deployment.

6. **Create a test config.** In config/site.js, change values to a fictional company: "Acme Therapeutics," different email, different slug.
7. **Create a free Supabase project** (supabase.com → New Project → "dataroom-test"). Run supabase-setup.sql.
8. **Deploy to Vercel.** Either create a new Vercel project from your branch, or use vercel CLI: vercel --prod. Set the env vars.
9. **Test the full flow:** Send yourself a magic link. Log in. View tabs. Check that analytics record in the new Supabase project. Verify notification emails.
10. **Check the admin page:** Log in with the GP_EMAIL you configured. Verify investor data appears.

If this all works, you've validated the pipeline. You can now confidently take on your first real client knowing the deployment process works.

What you can skip for the test deployment

Custom domain (just use the Vercel preview URL)

Resend custom domain (use your existing one)

The remaining component refactors (overview, memo, terms, model)

Admin page polish

These can be done before the first real client, but they're not needed to validate the pipeline.

Recommended Overall Sequence

Here's the full path from today to your first paying client:

Week	What to Do	Hours	Milestone
1	Session 1: Config extraction (MVP)	3	Config-driven deploy works locally
1	Session 2: Test deployment	3	Fictional client live on Vercel
2	Complete Phase 1: all components refactored	5–7	Zero hardcoded values remain
2	Phase 2: Setup script, deployment checklist	3–4	Repeatable deployment pipeline
3	Phase 3: Auth hardening, admin polish	6–8	Production-ready for external users
3	Write client onboarding doc + content template	2	Ready for first client intake
4	First real client deployment	4–6	Live client data room

Total: ~26–34 hours spread across 3–4 weeks, assuming ~8–10 hours per week. This is a conservative estimate that includes thorough testing.

Appendix A: Complete Hardcoded Values Inventory

Every company-specific value in the current codebase, with exact file and line references.

Value	File(s)	Occurrences
"CW Ventures"	app/pst/page.js, app/layout.js, lib/notifications.js	5
"Puget Sound Therapeutics"	app/pst/page.js, components/Shared.js	3
"pst" (deal slug)	app/pst/page.js, app/api/track/route.js, admin/page.js	4
360-318-4480	app/pst/page.js, components/CallSection.js, OverviewSection.js, FAQSection.js, Shared.js	5
camwiese@gmail.com	components/OverviewSection.js, .env.local (GP_EMAIL)	3
calendar.app.google/asdgEY3xfoLCNyWJ9	app/pst/page.js, components/CallSection.js	2
hello@ventures.cameronwiese.com	lib/notifications.js	2
/images/signature.png	components/OverviewSection.js	1
$25,000,000 (safe cap)	components/ModelSection.js, TermsSection.js, FAQSection.js	3+
$5,000,000 (round size)	components/TermsSection.js, FAQSection.js	2+
$35,000,000 (Series A raise)	components/ModelSection.js	1
$100,000,000 (Series A pre-money)	components/ModelSection.js	1
0.20 (carry)	components/ModelSection.js	1
2029 (exit year)	components/ModelSection.js	1
© 2026 CW Ventures	app/pst/page.js	1
Seed Round · SPV Data Room	app/pst/page.js	1
FAQ content (~50 Q&As)	components/FAQSection.js	All inline
Overview letter	components/OverviewSection.js	All inline
Deal memo content	components/MemoSection.js	All inline
Terms details	components/TermsSection.js	All inline
Science primer	components/ScienceSection.js	All inline
Interview description	components/InterviewSection.js	All inline

Appendix B: Client Onboarding Checklist

Send this to the client before starting their deployment. You need all of this before you can begin.

Company Information

- Company legal name (as it should appear in the data room)
- Deal name (e.g., “Seed Round,” “Series A,” “SPV”)
- Company logo (SVG or high-res PNG, transparent background)
- Favicon (32x32 PNG or SVG)
- Brand colors: primary dark, primary, accent (hex codes)

Founder Information

- Founder’s full name
- Founder’s email (this becomes the admin login)
- Founder’s phone number
- Calendar scheduling link (Calendly, Cal.com, Google Calendar, etc.)
- Signature image (PNG, transparent background) — optional

Content

- Executive summary / founder letter (1–2 pages, can be a Google Doc)
- Deal memo (the investment thesis, can be a Google Doc or PDF)
- Deal terms (instrument type, valuation cap, round size, check sizes)
- FAQ (questions and answers, organized by category)
- Financial model inputs: safe cap, round size, expected Series A raise, Series A pre-money, carry %, option pool %, exit year, exit scenarios
- Science primer or technical overview (if applicable)
- CEO interview video URL (if applicable, can be added later)

Domain & DNS

- Desired data room URL (e.g., dataroom.yourcompany.com)
- Access to DNS settings for the domain (or contact info for whoever manages DNS)
- Preferred sending email address for auth emails (e.g., noreply@yourcompany.com)

Access Control

- List of email addresses to exclude from analytics (founder + team members)

- Any additional admin email addresses (besides the founder)

Timing note for clients

Most of the setup time is waiting for you to provide content.
Once we have everything, your data room will be live within 48 hours.
Content can be rough drafts — we can iterate after the initial deploy.